LMU LUDWIG-MAXIMILIANS-UNIVERSITÄT MÜNCHEN
INSTITUT FÜR INFORMATIK
LEHRSTUHL FÜR DATENBANKSYSTEME
UND DATA MINING



Seminar Paper
in Modern Data Storage Technologies

Modern I/O Interfaces: Exploring the Potential of io_uring

Irina Comsa

Betreuer: Prof. Dr. Marcus Paradies and Assistant Constantin Pestka
Abgabedatum: 10.07.2026

Declaration of Authorship

I hereby declare that the thesis submitted is my own unaided work. All direct or indirect sources used are acknowledged as references.
This paper was not previously presented to another examination board and has not been published.

ORT, 10.07.2026

.....
Irina Comsa

Abstract

Modern NVMe SSDs deliver tens of millions of I/O operations per second, leaving traditional I/O interfaces incapable of exploiting these gains due to large per-operation overheads. io_uring, a modern asynchronous I/O interface, is a promising candidate for closing this gap. However, its capacity to saturate modern storage hardware compared to libaio and SPDK, along with the conditions for its success, remains understudied.

This paper addresses these two gaps by reviewing four recent studies on io_uring's performance with different system configurations, workloads, and optimizations.

The first key finding is that io_uring can saturate modern hardware under specific configurations, while libaio never achieves this performance. Though io_uring falls behind SPDK, the best-performing library, it preserves OS abstractions. Hence, io_uring is more suited for production database systems that rely on them than the user-space library SPDK.

The second key finding is that this paper identifies three conditions for achieving this performance: when workloads expose sufficient I/O parallelism, the system's architecture embraces io_uring's capabilities, and when optimizations target actual bottlenecks. io_uring falls short in CPU-constrained environments, memory-bound workloads, and scenarios with few I/O submissions. io_uring is not a drop-in replacement: effective deployment requires architectural redesign and workload-specific tuning.

Contents

1 Introduction **2**

2 Background **4**
2.1 Modern NVMe Storage Technologies 4
2.2 Modern Asynchronous I/O Interfaces 4
2.2.1 libaio 4
2.2.2 SPDK 5
2.2.3 io_uring 5

3 Recent Studies **8**
3.1 Performance in fio Micronbenchmark 8
3.2 Performance in a Modern Storage Engine Prototype 9
3.3 Performance in Storage- and Network-Bound Workloads 10
3.3.1 io_uring in a Storage-Bound Buffer Manager 11
3.3.2 io_uring in a Network-Bound Data Shuffle 12
3.4 Polling is Not Universally Better 13

4 Discussion **15**
4.1 RQ1: The Potential of io_uring for saturating modern NVMe storage 15
4.2 RQ2: io_uring's Conditions for Success and Failure 16

5 Conclusion and Outlook **18**
5.1 Acknowledgments 18

Bibliography **19**

Chapter 1

Introduction

Over the past decade, storage hardware has evolved dramatically. Modern non-volatile memory (NVM) storage technologies can sustain millions of I/O operations per second (IOPS) and bandwidths of hundreds of gigabits per second [7]. However, even storage engines specifically optimized for NVMe storage fall short of exploiting this hardware potential. A 2023 study by [5] reports a 3.5× gap between theoretical hardware capability and actual software performance. The bottleneck has shifted: it is no longer the hardware that limits performance but the software layer between the application and the storage device, the I/O interface.

Modern asynchronous I/O interfaces address this bottleneck by allowing applications to issue multiple requests concurrently without blocking, significantly increasing throughput and CPU efficiency compared to traditional synchronous interfaces [4]. However, a fundamental trade-off exists among modern solutions. Kernel-based APIs (Application Programming Interfaces) such as libaio incur system-call and context-switch overhead that prevents full NVMe saturation, while user-space frameworks such as SPDK (Storage Performance Development Kit) eliminate this overhead by bypassing the kernel entirely at the cost of OS abstractions such as file systems and access control that production databases depend on [7].

The `io_uring` interface, introduced into the Linux kernel in 2019, is considered a promising candidate for bridging the hardware-software gap. `io_uring` builds on established concepts from high-performance storage and networking such as asynchronous I/O, shared memory-mapped queues and polling modes [5]. It aims to deliver high performance while preserving kernel abstractions [7]. Despite `io_uring`'s potential to close the hardware-software gap, there has been little work on understanding how to adapt this API to the requirements of database systems [7] and how to reach its performance potential [6]. Hence, this seminar paper is guided by the following research questions:

- **RQ1: What is `io_uring`'s potential for saturating modern NVMe storage compared to libaio and SPDK?**
- **RQ2: Under what conditions does `io_uring` achieve its performance potential, and when does it fall short?**

To answer these questions, this paper is structured as follows. Chapter 2 provides background on NVMe and modern I/O interfaces, covering libaio, SPDK, and `io_uring`'s architecture and operating modes. Chapter 3 presents four studies published between 2022 and 2025 that evaluate `io_uring`'s performance under different workloads, configurations, and system designs. Chapter 4 analyzes the findings of these studies in the context of the research questions, and Chapter 5 summarizes the key conclusions and provides an outlook on future developments.

Chapter 2

Background

This Chapter provides the needed background on NVMe storage and the three modern asynchronous I/O libraries (libaio, SPDK and io_uring) to understand the studies reviewed in Chapter 3.

2.1 Modern NVMe Storage Technologies

With older, slower storage devices, the overhead of interrupt processing and context switching in traditional interrupt-based I/O was negligible in comparison to the storage device performance [11]. As NVMe SSDs (Non-Volatile Memory Express Solid State Drives) have become dramatically faster, this same overhead now accounts for up to 15% of total I/O latency [10, 9]. Modern NVMe SSDs such as the Samsung Z-SSD, Intel Optane SSD, and Toshiba XL-Flash have led to substantial advancements in I/O latency [8]. These storage technologies can support up to millions of I/O operations per second (IOPS) [4]. However, CPU performance improvements haven't kept pace over the past years [4].

2.2 Modern Asynchronous I/O Interfaces

2.2.1 libaio

libaio is a Linux kernel I/O Application Programming Interface (API) that supports asynchronous I/O using interrupt-based completion processing [4]. Compared to traditional blocking I/O APIs such as POSIX calls, libaio offers better performance, ease of use, and flexibility by allowing applications to issue multiple I/O operations without blocking [4]. The interface provides two system calls: **io_submit** for submitting I/O requests to the kernel and

io_getevents for retrieving the completed I/O requests from the kernel. The main drawback of libaio is the I/O performance overhead created by the reliance on two system calls per I/O operation, the performance overhead, as each system call triggers a context switch between user and kernel mode. Additionally, libaio copies metadata on every submission and completion, adding further per-request cost [4]. A further constraint is that libaio is restricted to unbuffered I/O access, requiring the use of O_DIRECT mode [4, 3].

2.2.2 SPDK

The Storage Performance Development Kit (SPDK) is an open-source framework consisting of libraries and tools for building high-performance user-space storage applications, introduced by Intel and since developed into a community-driven storage framework [13]. SPDK moves the block device driver into user space, eliminating the overhead of context switches and system calls associated with kernel-based I/O paths. Because it operates entirely outside the kernel, SPDK depends exclusively on polling to detect I/O completion and cannot make use of interrupt-driven completion [11]. This design allows SPDK to substantially outperform conventional kernel interfaces [4], but at the cost of exclusive device access and reduced integration with kernel-provided services such as file systems, access control, and scheduling.

2.2.3 io_uring

io_uring is an asynchronous I/O interface for the Linux kernel, described by its creator Jens Axboe for the first time in 2019 in the paper *"Efficient IO with io_uring"* [1]. It was designed to overcome the limitations of existing asynchronous I/O interfaces, aiming to retain the broad compatibility and flexibility of interfaces such as libaio while approaching the performance of user-space frameworks [4].

Memory-Mapped Ring Buffers. io_uring achieves performance through queue-driven communication between application and kernel via two ring buffers mapped into user-space memory: the Submission Queue (SQ) and the Completion Queue (CQ) [4, 7]. These shared ring buffers form the core of the io_uring architecture [1]. Both buffers are managed via head and tail pointers, allowing the application and kernel to coordinate access without locking. An application first calls `io_uring_setup` to create an io_uring instance and maps the resulting rings into its address space via `mmap` [1]. It may additionally call `io_uring_register` to pre-register resources such as buffers or file descriptors with the instance, reducing per-request setup costs. Because both queues are

placed in memory shared between user space and the kernel, submitting and completing requests avoids additional data copies and, depending on the configuration, it reduces the number of system calls needed to submit and reap I/O [4].

io_uring's Configuration Modes io_uring supports three principal configuration modes, each progressively eliminating software overhead to optimize I/O performance [4]. These modes, illustrated in Figure 2.1, offer distinct trade-offs between CPU efficiency and latency.

In the **default mode (iou)**, the application places one or more Submission Queue Entries (SQEs) at the SQ tail and notifies the kernel of pending requests with an `io_uring_enter` call, specifying via its `to_submit` parameter how many entries are ready for submission. Because multiple requests can be batched into a single call, this reduces the average number of syscalls per I/O operation compared to libaio, which always issues one `io_submit` call for submission and one `io_getevents` call to reap completions. `io_uring_enter` can additionally be used to wait for a specified number of completions in the same call, via its `min_complete` parameter and the `IORING_ENTER_GETEVENTS` flag, further reducing the number of syscalls needed per I/O operation [4]. Inside the kernel, the request passes through the same abstraction layers as other kernel I/O interfaces. Once the SSD signals completion via a hardware interrupt, the kernel writes a Completion Queue Entry (CQE) to the CQ tail, which the application reads directly.

The **completion polling mode (iou+p, IOPOLL)** uses the same submission path as the default mode but replaces interrupt-driven completion notification with polling directly on the CQ [4, 7]. This removes interrupt-handling overhead from the completion path.

The **submission polling mode (iou+k, SQPoll)** completely removes submission-path syscalls. A dedicated kernel worker thread continuously polls the SQ, submits requests to the device on the application's behalf, and posts completions to the CQ [4, 7]. When combined with completion polling, this configuration eliminates system calls from the I/O path entirely, at the cost of dedicating a CPU core to the kernel poller thread.

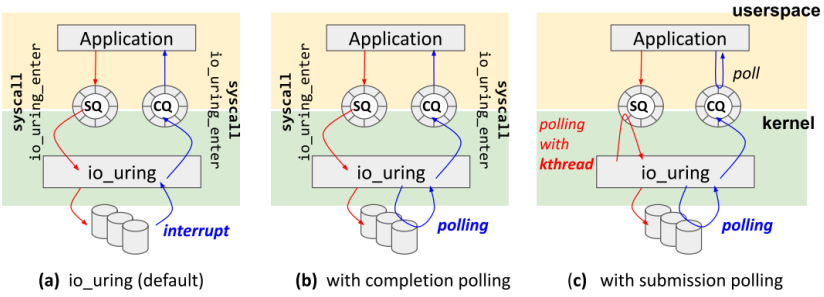


Figure 2.1: io_uring's configuration modes considered in [4]

Chapter 3

Recent Studies

To answer the research questions of this paper, this chapter reviews relevant insights from four recent studies published between 2022 and 2025 [4, 5, 7, 12]. These selected works provide a comprehensive evaluation of io_uring's performance under diverse configurations.

3.1 Performance in fio Micronbenchmark

Didona et al. (2022) [4] conducted the first systematic comparison of libaio, SPDK, and the three configuration modes of `io_uring` (default, IOPOLL and SQPOLL) using fio microbenchmark on raw block devices. By configuring fio to perform only random data reads at the granularity of 4KiB using unbuffered I/O, they maximized IOPS to isolate API-level overhead and evaluate scalability trends.

Evaluating the Effects of Polling on Performance To investigate the influence of polling on the performance, researchers initially ran a single fio job on one NVMe SSD drive with multiple queue depths (from 1 to 128) using one to two CPU cores located on the same NUMA node as the drive. The results [4, Figs. 2–3] report IOPS across different queue depths, the corresponding median latencies, and the average number of system calls per I/O operation.

iou+k (SQPOLL) was the only library highly sensitive to core availability. With a single CPU, it fell behind all other APIs by nearly an order of magnitude. This occurred because the fio and kernel poller threads competed for the same core, with the kernel thread consuming 50% of the cycles and delaying I/O. Adding a second core restored performance to within 18% of SPDK's throughput and 10% of its latency. To find its optimal CPU-to-drive ratio, 5 fio jobs (one per drive) ran at a fixed 128 depth across 5, 6, 7, and 10 cores. The results are illustrated by [4, Fig.4] and indicate that iou+k needs

one dedicated core per kernel polling to reach peak performance. At this optimal ratio, iou+k came within $\tilde{1}5\%$ of SPDK's throughput. However, with suboptimal core allocation, iou+k was the worst-performing library. Section 3.2 investigates whether this holds in a full storage engine.

Furthermore, SPDK delivered the best performance overall and was the only library capable of saturating the drive's bandwidth since it is not CPU-bound compared to the other approaches.

Lastly, several libraries showed similar performance depending on the number of queue depths. iou+p (IOPOLL) matched SPDK's performance at lower queue depths (up to 16) but fell behind at higher depths once system call overhead became a bottleneck. Similarly, iou and libaio performed almost identically up to depth 16, beyond which iou pulled ahead because its lower per-I/O syscall count made it more CPU efficient. iou and iou+p performed nearly identically at higher depths (above 16), but at lower depths iou+p offered significantly better throughput and latency than iou.

Evaluating Scalability To study the APIs' scalability, the number of NVMe SSDs varied from 1 to 20, assigning one fio job per drive at a queue depth of 128 with a ratio of two CPU cores per drive (up to 20 physical cores). The results [4, Fig. 5] show that SPDK was the best-performing library once again, while the runner-up varied with the number of jobs and cores available. libaio, iou, and iou+p scaled steadily, but iou+k was highly sensitive to the CPU-to-drive ratio: it ranked second up to 10 drives (using all 20 cores), then dropped to the worst performer by 14 drives, with iou+p taking over as runner-up, closely followed by iou.

Based on these results, the researchers concluded that performance depends on polling and low system call overhead. io_uring, though never surpassing SPDK, can approach its performance with careful tuning and design-aware configuration.

3.2 Performance in a Modern Storage Engine Prototype

While the previous study evaluated the interfaces under isolated microbenchmark conditions, Haas and Leis (2023) [5] extended this line of work by additionally evaluating their performance under real database workloads within a full storage engine prototype called LeanStore. Notably, this paper found the SQPOLL mode to be a poor fit for their design, as the cores dedicated to kernel polling threads could otherwise have been used by database worker

threads. Hence, they excluded it from further evaluation. Another key difference from the previous paper consists in the use of more performant SSDs and CPU cores.

Microbenchmark Consistent with the previous study, SPDK delivered the best overall performance [5, Fig. 6]. Using a custom I/O benchmarking tool, the authors measured throughput while varying thread count from 1 to 64. SPDK reached full bandwidth (12.5M IOPS) with just 3 threads. Unlike the previous study, however, SPDK wasn't the only interface to reach full bandwidth: IOPOLL came close at 16 threads and matched it at 32. Also consistent with the previous study, io_uring's default mode and libaio performed nearly identically and reached a maximum of 10M IOPS from 16 threads onward. io_uring's submission polling wasn't considered in this experiment.

Evaluation with Realistic Workload SPDK's superior performance was attributed to the greater CPU efficiency of user-space libraries compared to kernel-based approaches. To validate these findings, they evaluated the libraries using LeanStore, a more realistic environment than I/O microbenchmarks [5, Fig. 15 and 16]. In this experiment, they varied the number of CPU cores to illustrate the CPU overhead of kernel-based interfaces in a read-only lookup workload and a TPC-C workload. The performance discrepancy of SPDK compared to the kernel interfaces was more evident in the read-only workload than in TPC-C, as it is more I/O heavy. Here, SPDK was on average 60%-80% faster than kernel libraries. This discrepancy decreased in both workloads as the number of CPU cores increased.

Ultimately, the researchers credited the performance gap between kernel-based interfaces and SPDK to inefficiencies in the Linux kernel's I/O path implementation. Once the system went out of memory and actively used the SSD in the TPC-C workload, kernel interfaces spent 16%–19% of all CPU cycles only on I/O submission, while SPDK spent only $\tilde{2}$%. Therefore, the authors noted that ongoing kernel development may narrow this gap over time. Moreover, they advise on carefully co-designing the storage engine with the flash NVMe SSDs to fully exploit flash storage.

3.3 Performance in Storage- and Network-Bound Workloads

In a more recent study from 2025, Jasny et al. [7] presented the first analysis of `io_uring` across both storage- and network-bound workloads in database systems. These two complementary use cases allowed them to demonstrate

under which system conditions io_uring delivers the greatest benefits and how to use it effectively in modern database systems.

3.3.1 io_uring in a Storage-Bound Buffer Manager

As a first use case, Jasny et al. [7] integrated io_uring into a transactional buffer-managed storage engine on 8 NVMe SSDs with a 1 GB buffer pool to examine storage-bound workloads. [7, Fig. 5] illustrates the results under different buffer manager designs and I/O execution modes.

They first employed a YCSB workload, which under uniform updates and 4 KiB pages produces a roughly 70% page fault probability, creating a heavily I/O-bound setting. In a fully synchronous setup, io_uring's default mode had the same throughput as the synchronous interface POSIX since performance was bound directly to device latency.

A slight improvement in io_uring's performance appeared when *batched write submission* was introduced in the buffer manager's eviction path. Batching lowered submission overhead for eviction and exploited device-level parallelism by grouping multiple I/O requests into a single system call to amortize write latency. Batching resulted in a $\tilde{1}4\%$ performance improvement.

The first meaningful performance gain appeared when the buffer manager was redesigned for *asynchronous transaction execution* using Boost. fibers C++ library [2] for cooperative scheduling. This architecture change leveraged io_uring's completion-based model, leading to nearly an order of magnitude increase in io_uring's throughput. The system could now overlap computation with I/O, keeping the CPU active and shifting the performance bottleneck from the I/O latency to CPU overhead.

Extending the buffer manager's architecture with *adaptive read batching submission* improved CPU overhead in the submission path by reducing per-I/O cycle cost and exploiting device-level parallelism. This mechanism dynamically adjusted batch sizes based on the ratio of outstanding I/Os to waiting fibers before entering the kernel. Hence, this increased throughput by about 18%.

Tuning io_uring for the Storage Engine After redesigning the buffer manager's architecture around io_uring's batching and asynchronous execution, the researchers turned to io_uring's low-level tuning features to further reduce per-I/O overhead. Notably, they found that the effectiveness of these features depends heavily on the workload characteristics.

For the I/O-intensive YCSB workload, four io_uring optimizations progressively improved performance: registered buffers (eliminating per-request copies), NVMe passthrough (bypassing generic storage-stack abstractions),

completion polling (removing interrupt overhead), and submission polling (eliminating syscalls). Together, they raised throughput by roughly 3.4–3.5× over the batched baseline.

In the TPC-C workload [7, Fig. 6], these optimizations brought less significant improvements, especially in the mostly in-memory configuration, where most reads are served directly from memory. Because this configuration is largely compute-bound, there is little I/O-path overhead left for the optimizations to reduce. In fact, completion polling performed slightly worse than the interrupt-driven baseline, and SQPoll brought no benefit because polling wasted CPU cycles when I/O activity was low. In the out-of-memory setting, tuning had greater impact since here I/O activity was higher due to the increased cost of page loads and evictions.

Microbenchmarks for a Detailed Analysis While previously larger batches increased throughput, targeted microbenchmarkings revealed that for latency-sensitive systems, overly aggressive batching is actually counterproductive. A burst of requests can temporarily overload the hardware's queue. Additionally, exceeding certain block size thresholds triggered an automatic fallback to io_worker threads, reintroducing the very latency and CPU overhead that io_uring was designed to eliminate.

3.3.2 io_uring in a Network-Bound Data Shuffle

As a second use case, Jasny et al. [7] employed io_uring for data shuffling in a distributed analytics engine on 400 Gbit/s networks, representing network-bound workloads. Unlike the buffer manager, which was asynchronous at the beginning, the shuffle engine adopted an asynchronous design from the start because asynchronous I/O is standard practice in distributed engines. They used *ring-per-thread* architecture, colocating computation and I/O within the same thread to avoid synchronization overhead with dedicated I/O threads. [7, Fig. 11] illustrates the per-node egress bandwidth for a six node shuffle with different tuple sizes and [7, Fig. 12] its corresponding memory bandwidth.

To vary workload characteristics, the authors shuffled a fixed data volume across different tuple sizes: smaller tuples produced more probe table inserts, stressing the CPU, while larger tuples shifted the bottleneck toward memory bandwidth. The default implementation of io_uring failed to saturate the the 400 Gbit/s link rate regardless of the workload characteristic, peaking at only 30 GiB/s per node.

Tuning io_uring for the Shuffle Engine Building on this baseline, the researchers applied io_uring's network-specific optimizations to close the remain-

ing gap to link speed. *Zero-copy send* eliminated the copy between user-space and kernel buffers by transmitting directly from pinned memory, while *zero-copy receive* wrote incoming data directly into registered user-space memory. Enabling both together roughly halved memory bandwidth consumption per unit of network throughput. For 4 KiB tuples, this reduction in memory pressure was sufficient to saturate the 400 Gbit/s link with only 16 worker threads per node.

Microbenchmarks for a Detailed Analysis Consistent with the buffer manager findings, the effectiveness of network-specific optimizations depended on workload characteristics. For smaller tuples (64 bytes), zero-copy optimizations provided marginal gains because random memory accesses from hash table insertions dominated the bottleneck, masking memory bandwidth benefits. For larger tuples (4 KiB and above), zero-copy became highly effective, reducing memory bandwidth consumption by approximately 50% and enabling link saturation.

From these two use cases, Jasny et al.[7] derived four practical guidelines for using io_uring effectively in database systems and validated them through a case study improving PostgreSQL's io_uring backend by 11–15%. These guidelines help answering the research questions in Chapter 4.

3.4 Polling is Not Universally Better

A recent study by Li et al. (2025) [12] examined whether polling is inherently more performant than interrupts for I/O completion. Using a 4 KiB `fio` read workload on an Optane SSD, they compared SPDK against io_uring configured for both interrupts (default) and polling (IOPOLL), then broke down the latency of each approach.

They found that most of the overhead attributed to interrupts does not stem from the interrupt mechanism itself but from a kernel scheduling policy not optimized for fast storage devices: after issuing a request, the kernel puts the requesting thread to sleep and invokes the scheduler, but since no other task is runnable, it must additionally context-switch to the idle task before switching back once the I/O completes. After replacing this with an active-checking policy that keeps the thread awake instead, interrupt-based I/O performed nearly as well as polling.

The study also reports two further findings regarding polling. At the storage-subsystem level, polling cannot coordinate with thread scheduling, which causes high tail latency once multiple tasks share a core. At the file-

system level, CPU-efficient polling designs require isolating polling threads behind costly IPC, and their event-driven programming model limits multicore scalability.

Chapter 4

Discussion

This chapter answers the two research questions stated in Chapter 1 based on the findings from the four studies presented in Chapter 3.

4.1 RQ1: The Potential of io_uring for saturating modern NVMe storage

Modern NVMe storage can be saturated by io_uring, but only under specific architectural conditions and workloads that will be discussed in Subsection 4.2, and at the cost of CPU.

io_uring vs. SPDK SPDK was the best-performing and most CPU-efficient library across all studies. As Li et al. [12] showed, this advantage stems primarily from bypassing the kernel rather than from polling itself, since the kernel's default scheduling policy is poorly suited to fast storage devices.

Under the tight CPU budget in Didona et al. [4], SPDK was the only library to fully saturate the hardware. However, with more capable CPUs, Haas and Leis [5] showed that io_uring can also fully saturate the hardware, though it required roughly twice the thread count of SPDK. SPDK's advantage is therefore one of CPU efficiency, not of exclusive saturation capability. This efficiency comes at a cost: kernel bypass sacrifices OS abstractions such as file systems and TCP networking, and it cannot securely share disks and CPUs among tasks, making SPDK difficult to integrate into production databases that rely on these abstractions [7]. Because io_uring preserves them, it is the more broadly deployable choice for saturating modern SSDs.

io_uring vs. libaio Across the first three studies [4, 5, 7], io_uring matched or exceeded libaio, pulling ahead as I/O intensity and queue depth increased

thanks to its fully asynchronous design and lower per-I/O system call overhead. libaio could not saturate modern NVMe arrays in any study because its two system calls per I/O and interrupt-based completion exhaust CPU cycles.

Overall, this makes io_uring the most promising interface for saturating NVMe storage in production database systems.

4.2 RQ2: io_uring's Conditions for Success and Failure

As Jasny et al. [7] and Didona et al. [4] both remarked, naively adopting io_uring does not guarantee performance gains and can even cause regressions. Across the four studies, io_uring approaches or matches hardware saturation only when specific conditions align. This subsection identifies three such conditions based on the findings presented in Chapter 3 and on Jasny et al.'s four guidelines [7] for using io_uring effectively in database systems.

Three conditions under which io_uring achieves its potential:

1. **When the workload is I/O-intensive with sufficient parallelism to exploit io_uring's batching feature.** Batching amortizes per-I/O submission overhead only when enough concurrent requests are available. In Didona et al. [4], io_uring's default mode performed nearly identically to libaio at low queue depths up to 16 and only pulled ahead once it had a lower average system calls count than libaio. Jasny et al. [7] confirmed this pattern in the buffer manager use case, where io_uring provided meaningful gains only once the workload generated sufficient concurrent page faults to keep batches full.

2. **When the system's architecture embraces io_uring's capabilities.** Haas and Leis [5] achieved near-saturating throughput only after co-designing their storage engine with the flash NVMe SSDs to fully exploit flash storage. Jasny et al. [7] demonstrated most explicitly in the storage-bound workload how redesigning the architecture of the buffer manager to first embrace io_uring's batching feature and then its fully asynchronous execution can more than double the performance. Batching write operations improves performance by about 14%. In the shuffle engine, the ring-per-thread architecture improved performance by avoiding synchronization overhead and enhancing cache locality, both of which help sustain high throughput.

3. **When the io_uring's optimizations are carefully selected.** io_uring is a unified interface that offers general, storage- and networking-specific optimizations for I/O-intensive systems [7]. Knowing which optimizations to use and in which contexts can significantly improve io_uring's performance. In Didona et al.'s microbenchmark [4], SQPOLL reached second-best performance after SPDK only when a dedicated CPU per kernel polling thread was available.

When io_uring falls short Simply implementing io_uring without considering these three conditions might result in massive performance regressions.

In workloads that are not I/O-bound, such as Jasny et al.'s in-memory TPC-C, where polling wastes CPU cycles on sparse I/O activity [7] io_uring's polling configurations resulted in marginal or even negative gains.

A system architecture that does not facilitate io_uring's capabilities, such as the initial fully synchronous buffer manager setup in [7], led to io_uring performing as badly as an asynchronous interface like POSIX.

Third, io_uring's own tuning features can backfire when applied without regard for system limits. io_uring underperforms when CPU resources are scarce, as in Didona et al.'s single-core setup where SQPOLL fell to an order of magnitude below libaio [4]. This sensitivity to CPU-to-drive motivated Haas and Leis to exclude this configuration from their further evaluations [5]. In Jasny et al. study [7] overly aggressive batching introduced tail-latency spikes and exceeded block-size thresholds, which triggered a fallback to io_worker threads, reintroducing the very overhead that io_uring was designed to eliminate [7].

These failure patterns share a common cause: io_uring's gains are proportional to the fraction of workload cost that lies on the I/O path. When that fraction is low, no amount of tuning can compensate. io_uring is therefore not a drop-in replacement but requires a thorough understanding of its design and the target workload.

Chapter 5

Conclusion and Outlook

This paper investigated io_uring's potential for saturating modern NVMe storage and the conditions under which this potential is realized. Through a review of four recent papers evaluating io_uring's performance potential, two main conclusions were reached.

First, io_uring can saturate modern NVMe SSDs, while libaio can not achieve this performance. Though SPDK is more CPU-efficient, io_uring preserves kernel abstractions such as file systems and networking, making it the more deployable choice for production database systems.

Second, io_uring is not a drop-in replacement. It achieves its potential only when workloads expose sufficient I/O parallelism, systems are redesigned for asynchronous execution, and optimizations are tuned to workload-specific bottlenecks. Otherwise, particularly under CPU pressure or in compute-bound workloads, it provides marginal benefit and can even underperform simpler interfaces.

Outlook. io_uring is under active development, with recent additions such as zero-copy receive continuing to expand its capabilities. Ongoing kernel improvements may also narrow the CPU efficiency gap with SPDK. Existing studies focus largely on synthetic benchmarks and prototype engines. Evaluating io_uring in production database systems beyond PostgreSQL remains an open direction.

5.1 Acknowledgments

The author acknowledges the use of AI for language refinement and stylistic improvements. All ideas, analysis, and conclusions are the author's own and/or are based on the referenced papers.

Bibliography

[1] Jens Axboe. Efficient IO with io_uring. https://kernel.dk/io_uring.pdf. Accessed: 2026-05-25.

[2] Boost Orgs. Boost.fiber: C++ cooperative user-mode threads. https://github.com/boostorg/fiber. Accessed: 2026-07-10.

[3] Jonathan Corbet. The rapid growth of io_uring. https://lwn.net/Articles/810414/. Accessed: 2026-05-25.

[4] Diego Didona, Jonas Pfefferle, Nikolas Ioannou, Bernard Metzler, and Animesh Trivedi. Understanding modern storage apis: a systematic study of libaio, spdk, and io_uring. In *Proceedings of the 15th ACM International Conference on Systems and Storage*, SYSTOR '22, page 120–127, New York, NY, USA, 2022. Association for Computing Machinery.

[5] Gabriel Haas and Viktor Leis. What modern nvme storage can do, and how to exploit it: High-performance i/o for high-performance storage engines. *Proc. VLDB Endow.*, 16(9):2090–2102, May 2023.

[6] Brynjar Ingimarsson. Exploring the performance of the io_uring kernel i/o interface. Master's thesis, Vrije Universiteit Amsterdam and Universiteit van Amsterdam, Amsterdam, Netherlands, June 2024. Available at https://atlarge-research.com/pdfs/2024-bingimarsson-msc_thesis.pdf.

[7] Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, Viktor Leis, and Carsten Binnig. High-performance dbmss with io$_u$*ring* : *Whenandhowtouseit*, 2025.

[8] Sungjoon Koh, Junhyeok Jang, Changrim Lee, Miryeong Kwon, Jie Zhang, and Myoungsoo Jung. Faster than flash: An in-depth study of system challenges for emerging ultra-low latency ssds. In *2019 IEEE International Symposium on Workload Characterization (IISWC)*, pages 216–227, 2019.

[9] Sungjoon Koh, Changrim Lee, Miryeong Kwon, and Myoungsoo Jung. Exploring system challenges of ultra-low latency solid state drives. In *Proceedings of the 10th USENIX Conference on Hot Topics in Storage and File Systems*, HotStorage'18, page 15, USA, 2018. USENIX Association.

[10] Gyusun Lee, Seokha Shin, and Jinkyu Jeong. Efficient hybrid polling for ultra-low latency storage devices. *Journal of Systems Architecture*, 122:102338, 2022.

[11] SeulA Lee and Jiwoong Park. Comparative performance analysis of i/o interfaces on different nvme ssds in a high cpu contention scenario. *IEICE TRANSACTIONS on Information*, E107-D(7):898–900, July 2024.

[12] Chuandong Li, Ran Yi, Zonghao Zhang, Jing Liu, Changwoo Min, Jie Zhang, Yingwei Luo, Xiaolin Wang, Zhenlin Wang, and Diyu Zhou. Aeolia: A fast and secure userspace interrupt-based storage stack. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles*, SOSP '25, page 479–495, New York, NY, USA, 2025. Association for Computing Machinery.

[13] Ziye Yang, James R. Harris, Benjamin Walker, Daniel Verkamp, Changpeng Liu, Cunyin Chang, Gang Cao, Jonathan Stern, Vishal Verma, and Luse E. Paul. Spdk: A development kit to build high performance storage applications. In *2017 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pages 154–161, 2017.